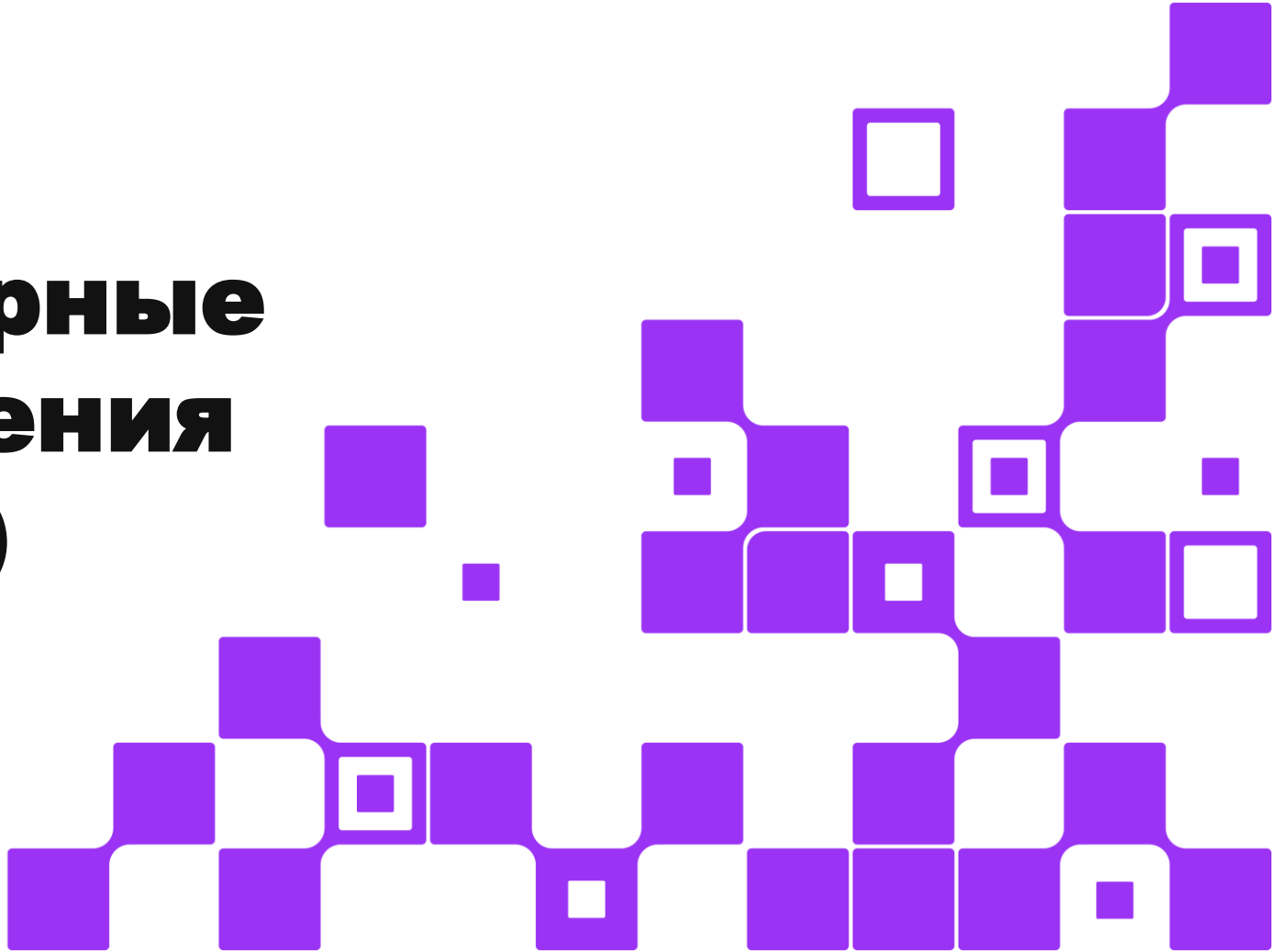
IThub

Регулярные выражения (Regex)



IThub

Если у вас есть проблема и вы решили использовать регулярки, то теперь у вас две проблемы.

IThub

Регулярные выражения (Regular Expressions, RegEx) - это специальный язык для описания шаблонов поиска в строках.

искать нужный текст

проверять соответствие строки определённому формату

извлекать части строки

разбивать строку на части

- проверка email
- проверка номера телефона
- поиск слов в тексте
- поиск URL

- проверка пароля
- извлечение дат
- замена частей текста
- разбор логов

Основные специальные символы

IThub

Символ	Значение
.	любой символ
\d	цифра
\D	не цифра
\w	буква, цифра или _
\W	не \w
\s	пробельный символ
\S	не пробельный символ
^	начало строки
$	конец строки

Например:

\d\d

найдёт две цифры

подряд

Квантификаторы

IThub

Квантификатор	Значение
*	0 или более
+	1 или более
?	0 или 1
{n}	ровно n раз
{n,}	n или более
{n,m}	от n до m

Например:

\d+ означает: одна или более цифр

\d{4} означает: ровно четыре цифры

Наборы символов

IThub

квадратные скобки

[abc] означает: один символ: a, b или c.

диапазоны

[a-z] буквы от a до z
[0-9] цифры от 0 до 9

можно объединять диапазоны

[a-zA-Z0-9] буквы и цифры

Альтернативы и группировка

IThub

символ | означает «или»

cat|dog найдёт: cat или dog

круглые скобки позволяют группировать выражения

(cat|dog)

? означает, что символ может присутствовать или отсутствовать

https?:// означает: http:// или https://

Экранирование в Java

IThub

Здесь появляется важная особенность Java.
В регулярном выражении: \d означает цифру.
Но в Java строка должна записываться: "\\d"

Почему?
Потому что \ используется одновременно:

- в синтаксисе Java-строк
- в синтаксисе регулярных выражений

Поэтому: "\\d+"передаёт регулярному выражению: \d+

Метод String.matches()

IThub

Самый простой способ проверить строку

```
String text = "12345";

boolean result = text.matches("\\d+");

System.out.println(result);
```

```
true
```

Класс Pattern

IThub

java.util.regex.Pattern для более сложной работы.
Сначала создаём шаблон:

```
Pattern pattern = Pattern.compile("\\d+");
```

Затем создаём объект Matcher:

```
Matcher matcher = pattern.matcher("У меня 2 яблока");
```

И можем искать совпадения:

```
while (matcher.find()) {
    System.out.println(matcher.group());
}
```

IThub

Pattern

Компилирует регулярное выражение и создаёт шаблон для поиска.

```
Pattern pattern =
    Pattern.compile("\d+");
```

Шаблон можно использовать многократно для разных строк.

Matcher

Применяет шаблон к конкретной строке и ищет совпадения.

```
Matcher matcher =
    pattern.matcher(text);
```

Matcher хранит состояние поиска и предоставляет методы для работы с результатами.

find() / matches() / group()

Методы для поиска, проверки и извлечения данных из строки.

```
matcher.find();
matcher.matches();
matcher.group();
```

Позволяют найти совпадения, проверить соответствие всей строки и получить найденные фрагменты.

find() matches() group() - в чём разница?

IThub

find() — Ищет следующее совпадение

- Находит следующий фрагмент строки, который соответствует шаблону.
- Можно вызывать несколько раз, чтобы найти все совпадения.
- Возвращает **true**, если найдено совпадение, и **false**, если больше совпадений нет.

Пример

```
String text = "В строке 12 яблок и 45 груш";
Pattern pattern = Pattern.compile("\d+");
Matcher matcher = pattern.matcher(text);

while (matcher.find()) {
    System.out.println(matcher.group());
}
```

Результат

```
12
45
```

`find()` нашёл два совпадения: 12 и 45.
Каждый вызов `find()` перемещает поиск на следующее совпадение.

matches() — Проверяет всю строку целиком

- Проверяет, соответствует ли вся строка шаблону.
- Возвращает **true**, если вся строка полностью соответствует регулярному выражению.
- Если в строке есть лишние символы — возвращает **false**.

Пример

```
String text1 = "12345";
String text2 = "abc123";

Pattern pattern = Pattern.compile("\d+");

boolean result1 = text1.matches("\d+");
boolean result2 = text2.matches("\d+");
```

Результат

```
result1 = true
result2 = false
```

`matches()` подходит, когда нужно проверить формат всей строки (например, email, номер телефона и т. д.).

group() — Извлекает найденные данные

- Возвращает найденный фрагмент строки.
- Работает только после успешного вызова `find()` или `matches()`.
- Позволяет получить группы, выделенные в круглых скобках.
- `group(0)` — всё найденное совпадение. `group(1)`, `group(2)` и т. д. — содержимое групп.

Пример

```
String text = "Дата: 18.09.2026";
Pattern pattern = Pattern.compile("(\d{2})\.(\d{2})\.(\d{4})");
Matcher matcher = pattern.matcher(text);

if (matcher.find()) {
    System.out.println("День: " + matcher.group(1));
    System.out.println("Месяц: " + matcher.group(2));
    System.out.println("Год: " + matcher.group(3));
}
```

Результат

```
День: 18
Месяц: 09
Год: 2026
```

`group()` позволяет получить конкретные части совпадения, выделенные в выражении круглые скобки.

Извлечение данных с помощью групп

IThub

Круглые скобки позволяют создать группу.

Например: (\d{2})\.(\d{2})\.(\d{4})
может разобрать дату: 18.09.2026
На группы:
18 → день
09 → месяц
2026 → год

```
Pattern pattern =
        Pattern.compile("(\\d{2})\\.(\\d{2})\\.(\\d{4})");

Matcher matcher = pattern.matcher("18.09.2026");

if (matcher.find()) {
    int day = matcher.group(1);
    int month = matcher.group(2);
    int year = matcher.group(3);
}
```

Замена текста

IThub

Регулярные выражения можно использовать не только для поиска, но и для замены.

```
String text = "Цена: 100 рублей";

String result = text.replaceAll("\\d+", "***");

System.out.println(result);
```

```
Цена: *** рублей
```

Разбиение строки

IThub

Метод split() также поддерживает регулярные выражения.

```
String text = "Java,Python,C++";

String[] languages = text.split(",");
```

```
Java
Python
C++
```

Задание

IThub

Анализ логов сервера

```
String log = """
  [INFO] User alice logged in from 192.168.1.10 email=alice@gmail.com
  [ERROR] User bob failed login from 10.0.0.15 email=bob.smith@yahoo.com
  [INFO] User charlie logged in from 172.16.0.5 email=charlie@company.ru
  [ERROR] User alice failed login from 192.168.1.10 email=alice@gmail.com
  """;
```

1) Найти все IP-адреса
С помощью Matcher.find() найдите все IP-адреса.
Результат:

192.168.1.10
10.0.0.15
172.16.0.5
192.168.1.10

2) С помощью отдельного Regex найдите все email-адреса (find()).
Результат:

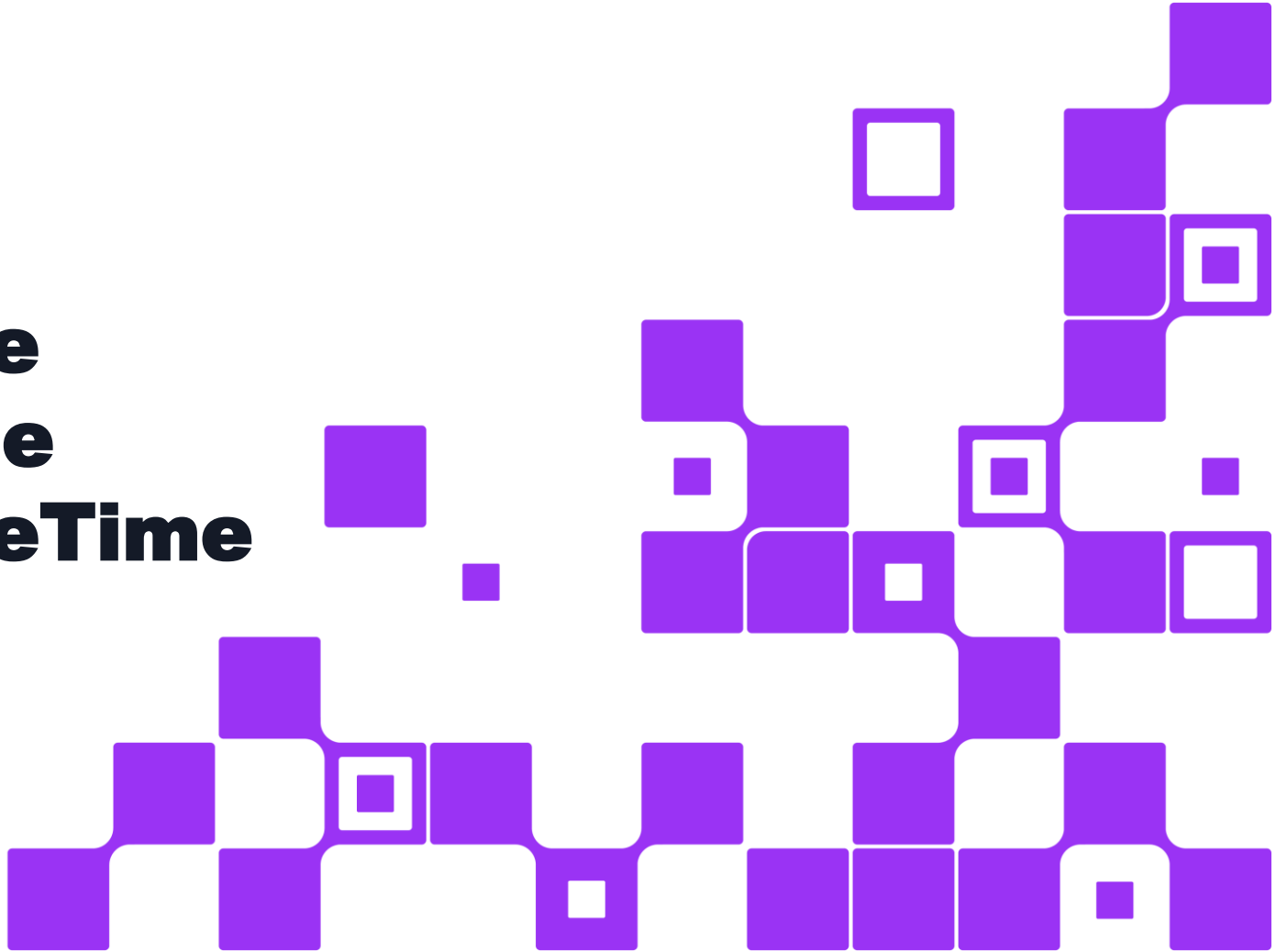
alice@gmail.com
bob.smith@yahoo.com
charlie@company.ru
alice@gmail.com

3) Составьте Regex, который одновременно захватывает: имя пользователя и IP-адрес (через group()).
Для каждого совпадения выведите:

User: alice, IP: 192.168.1.10
User: bob, IP: 10.0.0.15
User: charlie, IP: 172.16.0.5
User: alice, IP: 192.168.1.10

IThub

Date
LocalDate
LocalTime
LocalDateTime



IThub

В Java существует несколько способов работы с датой и временем.
Исторически первым появился класс Date, но со временем он оказался неудобным и был заменён более современным API из пакета java.time.

Класс LocalDate может содержать только дату.
Например, 2021-02-28

Класс LocalTime может содержать только время.
Например, 19:32:25.457826

Класс LocalDateTime содержит как дату, так и время.
Например, 2021-02-28T19:32:25.457826

Класс Date

IThub

✅ простой в использовании
✅ поддерживается большим количеством старого кода

❌ многие методы устарели (deprecated)
❌ неудобное форматирование
❌ проблемы с часовыми поясами
❌ изменяемый объект (mutable)

Пример

```
Date date = new Date();

date.setYear(130);
```

IThub

В Java 8 появился новый пакет java.time

LocalDateтолько дата

LocalTime только время

LocalDateTime дата + время

ZonedDateTime дата + время + часовой пояс

Главная идея нового API - разделить дату и время на отдельные понятия.

Класс LocalDate

IThub

LocalDate хранит только дату: год-месяц-день

```
LocalDate today = LocalDate.now();

System.out.println(today);
```

```
2026-09-20
```

Создание даты вручную:

```
LocalDate birthday =  LocalDate.of(1998, 07, 24);
System.out.println(birthday);
```

```
1998-07-24
```

Класс LocalDate

IThub

LocalDate хранит только дату: год-месяц-день

```
LocalDate today = LocalDate.now();

System.out.println(today);
```

```
2026-09-20
```

Создание даты вручную:

```
LocalDate birthday =  LocalDate.of(1998, 07, 24);
System.out.println(birthday);
```

```
1998-07-24
```

date.getYear() – вернет год

date.getMonth() – вернет месяц

date.getDayOfMonth() – вернет день

Методы LocalDate

IThub

```
date.plusDays(1);
```

```
date.plusMonths(1);
```

```
date.plusYears(1);
```

```
date.isLeapYear();
```

```
date.getDayOfYear;
```

```
date. getDayOfWeek();
```

Важно:
LocalDate является **неизменяемым** объектом.
Методы возвращают новый объект.

true, если год високосный

Возвращает день года. Например, 59

Возвращает название дня недели. Например, SUNDAY

Класс LocalTime

IThub

LocalTime используется для хранения времени без даты.
Хранит:

- часы
- минуты
- секунды
- наносекунды

```
LocalTime time = LocalTime.now();

System.out.println(time);
```

```
13:45:20.123
```

Создание времени вручную:

```
LocalTime meeting = LocalTime.of(14, 30);
System.out.println(meeting);
```

```
14:30
```

Методы LocalTime

IThub

time.now() | time.getHour() | time.getMinute()	получить время
isAfter(LocalTime other) | isBefore(LocalTime other)	проверяет, идет ли данное время **после | до** указанного
compareTo(LocalTime other)	сравнивает два времени возвращает 0, если равны, положительное число, если исходное больше **other**, отрицательное, если меньше
plusHours(long hours)| minusHours **plusMinutes(long minutes) | minusMinutes** **plusSeconds(long seconds) | minusSeconds** **plusNanos(long nanos) | minusNanos**	Важно: LocalTime является **неизменяемым** объектом. Методы возвращают новый объект.

Класс LocalDateTime

IThub

LocalDateTime объединяет: дату и время. Используется, когда нужно хранить полный момент без часового пояса.

```
LocalDateTime now = LocalDateTime.now();

System.out.println(now);
```

```
2026-09-20T13:45:30
```

Создание даты и времени вручную:

```
LocalDateTime event = LocalDateTime.of( 2026, 12, 31, 23, 59 );
```

```
2026-12-31T23:59
```

Форматирование даты и времени

IThub

Для форматирования используется **DateTimeFormatter**

```
LocalDate date = LocalDate.now();

DateTimeFormatter formatter =
        DateTimeFormatter.ofPattern(
            "dd.MM.yyyy"
        );

String result = date.format(formatter);

System.out.println(result);
```

```
20.09.2026
```

Класс Duration

IThub

```
LocalDateTime start = LocalDateTime.of(2026, 10, 1, 10, 30);
LocalDateTime end = LocalDateTime.of(2026, 10, 1, 14, 45);

Duration duration = Duration.between(start, end);

System.out.println(duration);
```

Duration — это именно время, обычно часы/минуты/секунды.

```
PT4H15M
```

PT4H15M означает 4 часа 15 минут

```
duration.toHours();     // 4
duration.toMinutes();  // 255
duration.toSeconds(); // 15300
```

Класс Duration

IThub

Можно получить отдельные части:

```
duration.toHoursPart();      // 4
duration.toMinutesPart();   // 15
duration.toSecondsPart();  // 0
```

Класс Period

IThub

Period — это класс java.time, который представляет календарный промежуток времени: годы, месяцы и дни.

```
LocalDate start = LocalDate.of(2020, 5, 10);
LocalDate end = LocalDate.of(2026, 10, 2);

Period period = Period.between(start, end);

System.out.println(period);
```

```
P6Y4M22D
```

То есть 6 лет, 4 месяца, 22 дня.

Класс Period

IThub

Можно получить отдельные части:

```
period.getYears();     // 6
period.getMonths(); // 4
period.getDays();     // 22
```

IThub

	Period	Duration
Для чего	Календарное время	Точное время
Единицы	годы, месяцы, дни	дни, часы, минуты, секунды
Обычно работает с	LocalDate	LocalDateTime
Пример	«через 3 месяца»	«через 2 часа 30 минут»

Задание

IThub

У детективного агентства есть список событий расследования:

Опрос свидетеля	2026-10-01	09:30
Получение видеозаписи	2026-10-01	14:15
Анализ видеозаписи	2026-10-02	10:00
Опрос подозреваемого	2026-10-04	16:45
Передача дела прокурору	2026-10-07	12:00

```
public class InvestigationEvent {
    private String description;
    private LocalDate date;
    private LocalDateTime timestamp;

    // constructor, getters
}
```

Реализовать логику в классе:

```
public class Investigation {

    private List<InvestigationEvent> events;

    public void addEvent(InvestigationEvent event) {
        // добавить событие через конструктор
    }

    public List<InvestigationEvent> getEventsForDate(LocalDate date) {
        // получить события за конкретный день
    }

    public Duration getInvestigationDuration() {
        // От первого события до последнего
    }

    public List<InvestigationEvent> getEventsBetween(
            LocalDateTime from,
            LocalDateTime to) {
        // Должны попасть события, произошедшие в заданном временном диапазоне
    }
}
```